



AMUZA FC-90 SDK



Moshe Kashlinsky

kashlinskymoshe@gmail.com



Objective

- Decipher FC-90 Signals
- Connect to FC-90 via Python
- Be able to send and receive commands from the FC-90
- Create a SDK to interact and automate the FC-90

Requirements

- Linux OS (Tested on Raspberry Pi 3B+, running Raspbian)
- PyBluez library
- Python3

Challenges Faced

- Connecting to the device was a challenge. It came with an android app, which could be used with either USB or Bluetooth. USB did not work, so I was left with Bluetooth as an entry point
- Once I connected to it, I was able to send commands and then analyze the logs via Bluetooth. This helped me decipher most signals
- The biggest challenge was connecting to the device with my laptop. I initially tried via Windows, but all the libraries there only worked for BLE devices. As the FC90 is a Bluetooth device as opposed to BLE, this forced me to switch to Linux. I got a Raspberry Pi 3B+ and was able to finally connect to it that way
- Once I connected, there was a challenge of how to deal with the simultaneous processes. This was solved via threading.

Signals

- Signals sent in L2CAP format
- L2CAP is a standard Bluetooth protocol - this presented issues as most libraries are built for BLE (Bluetooth Low Energy) protocols
- Spent several days sending test commands via the app, then using Wireshark to decipher the signals
- A comprehensive list of signals is on the next slide

List of Commands

Preface: All commands start with @ and end with \n

- @Q\n - sent, requests an update from the machine
- @q\n - received, reports back the update from the machine
- @E\n - Exit code, sent when an operation is received by the machine
- @P,M<#>,XXXX,<Well List>\n\n - Sent, a move command in the format @P then defined a sequence of methods
- @T\n - Sent, a command that tells the machine to stop what it is doing
- @Y\n - Sent, ejects the tray
- @Z\n - Sent, inserts the tray
- @?\n - Gets current OS version of the AMUZA
- Needs to be run at the start
- @N\n - Sent, initializes the needle adjustment mode
 - To exit this mode, send @V,180\n, and then @T\n after waiting ~0.2 seconds
- @U01\n - Sent, Moves the needle up by 1 increment. Only works in Needle Adjustment mode
- @D01\n - Sent, moves the needle down by 1 increment. Only works in Needle Adjustment mode
- @V,<XXX> - Sent, sets the bed temperature to XX.X degrees celsius
- @Z\n - Sends the needle to the home position

Structure of Methods and Sequences

A method is an instruction that contains the list of wells the machine is supposed to visit, as well as the time spent at each well. A method is formatted as follows:

```
M<ID #>,<4 digit time>,<List of wells>,\n\n
```

One can put several methods together into a sequence, which looks as follows:

```
@P,<Method 1>,<Method 2>,\n\n
```

The Method and Sequence classes deal with the formatting for you, and all you need to do is input a list and time for the method, or a list of methods for the sequence

AmuzaConnection Class

The AmuzaConnection class is the main class that deals with connecting to the device. It has the following structure:

- Threads to handle asynchronous processes such as receiving responses from the machine
- A console UI that lets the user interact with the machine directly
- A function that handles connection with the device
- Callable functions that let the user directly send commands via the script
- A logging feature that outputs the user's and devices actions to a log file

The next slides will focus on each bullet here in depth

Initializing the script

```
if __name__ == '__main__':  
    connection = AmuzaConnection()  
    connection.connect()  
    connection.consoleInterface()
```

A typical, simple initialization of the script

The typical workflow with the script begins with the instantiation of an `AmuzaConnection` object. During instantiation, a log file is created, but not much else is done.

Once the log is created, you need to manually connect to the device by calling the `.connect()` function. This function searches for ~15 seconds and records all devices that it can connect to (excluding BLE devices)

It tries to connect to a device named “FC90-0037”, but if it fails to, then the connection fails to happen. User input is needed to confirm the connection typically.

Once it connects, it sends a few commands such as centering the needle, then sets up threads to send queries and receive responses from the device.

Typically, one would then call `.consoleInterface()`, which initializes a console UI for the user to use

```
19 Rcvd "@q,01,"  
34 Rcvd "0,000,0000,193,210,0\n"
```

A wireshark screenshot of a typical example of a single command split into two messages

Threads

There are two main threads in the script: `queryThread()` and `receptionThread()`

`queryThread()` simply sends a query command once every second. Think of it as the “heartbeat” of the connection

`receptionThread()` is constantly listening for output from the device. Sometimes, the output received does not all fit into one message, and it gets split into two. The job of this thread is to detect when one message starts and ends, chunk the pieces into one concrete message and pass it along to a handler function, `handleReceived(cmd)`. It does this by knowing that each message starts with `@`, and ends with `\n`

There is one more thread, `loopThread(threadEvent, sequence)`, which serves to loop a movement command. It will be addressed later

Reception Handling

There are two typical messages we can get. After we send a command, we get back @E,<Exit code>\n which tells us whether the command went through successfully or not. The handling function simply logs the exit code without much further action.

The other action is the response from the queries. These messages hold several key pieces of information, and are structured as follows:

@q,<device state>,<device moving?>,<current well location>,<time left at current well>,<unknown (did not figure out)>,<current temperature>

The function prints the current time at the current well IF the machine is moving, as well as updates the current state of the device based on the known states dictionary (defined at the top of the script)

Sometimes, we also get garbage from the machine. This is handled in the script by marking it in the log

Console UI

The script has a robust console UI that is easily initializable by the user. By calling `.consoleInterface()`, the console enters into a loop where the user enters some (case sensitive) commands. There are many, mostly self explanatory commands, which can be accessed with the HELP command.

The most complex is the MOVE command, which constructs a sequence to send to the machine. One can also choose to loop through the sequence, having the machine keep going effectively forever if one desires.

Acknowledgements

Thank you to all those who helped me with this project, especially:

- Shaun Regenbaum
- Noah Bernten